



UNIVERZITET U SARAJEVU
ELEKTROTEHNIČKI FAKULTET
ODSJEK ZA AUTOMATIKU I ELEKTRONIKU

Laboratorijska vježba 6

Praktikum iz logičkog dizajna i upravljanja zasnovan na FPGA razvojnom sistemu Gowin 9k

Tarik Ćatić

U ovoj vježbi implementira se diskretni PI regulator u SystemVerilogu, a ispravnost implementacije se rigorozno verifikuje simulacijom putem Verilator alata.

Teorijski uvod – diskretni PID regulator

Za razliku od prethodnih vježbi, u kojima je FPGA korišten za obradu i prikaz izmjerene analogne veličine, u ovoj vježbi se FPGA po prvi put koristi za implementaciju algoritma automatskog upravljanja – diskretnog PID (proporcionalno-integraciono-derivacijskog) regulatora. Kontinualni oblik PID regulatora, koji povezuje grešku $e(t)$ (razliku između željene i trenutne vrijednosti regulisane veličine) sa upravljačkom veličinom $u(t)$, dat je izrazom:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (1)$$

gdje je K_p koeficijent proporcionalnog djelovanja, K_i koeficijent integracijskog djelovanja, a K_d koeficijent derivacijskog djelovanja. Proporcionalni član reaguje na trenutnu grešku, integracijski član akumulira prošle vrijednosti greške i eliminiše stacionarnu grešku, dok derivacijski član reaguje na brzinu promjene greške.

Diskretizacija i inkrementalni (rekurzivni) algoritam

FPGA, za razliku od analognog kola, radi isključivo u diskretnim vremenskim trenucima određenim taktnim signalom, pa je izraz (1) potrebno diskretizovati. Integral se aproksimira sumom, a izvod razlikom unazad, u trenucima $t = kT$, gdje je T perioda odabiranja. Direktna (pozicijska) diskretizacija zahtijeva pamćenje greške u svim prethodnim trenucima zbog sumacionog (integracijskog) člana, što je za praktičnu hardversku realizaciju nepogodno. Ovaj problem se rješava prelaskom na inkrementalni (rekurzivni) oblik algoritma, u kojem se nova upravljačka vrijednost računa kao zbir prethodne vrijednosti i priraštaja $\Delta u(kT)$ [1]:

$$\Delta u(kT) = K_p [e(kT) - e((k-1)T)] + K_i T e(kT) + \frac{K_d}{T} [e(kT) - 2e((k-1)T) + e((k-2)T)] \quad (2)$$

$$u(kT) = u((k-1)T) + \Delta u(kT) \quad (3)$$

Grupisanjem članova uz $e(kT)$, $e((k-1)T)$ i $e((k-2)T)$ u izrazu (2), inkrementalni algoritam se može zapisati u tzv. paralelnom obliku, pogodnom za direktnu hardversku implementaciju [1]:

$$u(kT) = u((k-1)T) + K_1 e(kT) + K_2 e((k-1)T) + K_3 e((k-2)T) \quad (4)$$

$$K_1 = K_p + K_i T + \frac{K_d}{T}, \quad K_2 = -K_p - 2\frac{K_d}{T}, \quad K_3 = \frac{K_d}{T} \quad (5)$$

Prednost izraza (4) je u tome što se u svakom taktu pamte samo tri prethodne vrijednosti ($u((k-1)T)$, $e((k-1)T)$, $e((k-2)T)$), umjesto cjelokupne historije greške, što ga čini pogodnim za implementaciju u konačnom automatu sa ograničenom memorijom – kao što je FPGA.

Pojednostavljenje na PI regulator

Ukoliko se u sistemu ne koristi derivacijsko djelovanje (npr. zbog osjetljivosti D člana na šum mjerenja), postavlja se $K_d = 0$, čime se dobija proporcionalno-integracioni (PI) regulator. Iz izraza (5) tada slijedi $K_3 = 0$, pa treći član u (4) u potpunosti otpada, a preostala dva koeficijenta se svode na:

$$K_1 = K_p + K_i T, \quad K_2 = -K_p \quad (6)$$

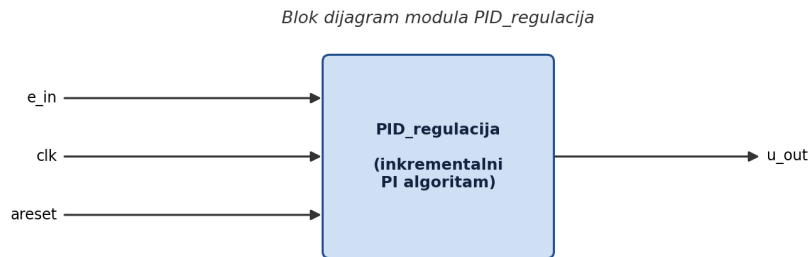
$$u(kT) = u((k-1)T) + K_1 e(kT) + K_2 e((k-1)T) \quad (7)$$

U ovoj vježbi implementira se upravo ovaj, pojednostavljeni PI slučaj, uz period odabiranja $T = 1$ s (odabran radi jednostavnije cjelobrojne aritmetike u hardveru – vremenska dimenzija se u tom slučaju "krije" unutar odabranih vrijednosti K_1 i K_2).

Struktura projekta

Za razliku od prethodnih vježbi, koje su zahtijevale fizičko povezivanje perifernih modula (ADS1115, džojstik) na Tang Nano 9K razvojni sistem, ova vježba je u potpunosti simulaciona: ispravnost implementiranog algoritma se provjerava isključivo softverskom simulacijom digitalnog kola, bez potrebe za fizičkim hardverom. Projekat se sastoji od jednog modula `PID_regulacija.sv`.

Blok dijagram modula, sa njegova tri ulaza i jednim izlazom, dat je na slici 1.



Slika 1: Blok dijagram modula PID_regulacija.

Ulaz e_in predstavlja trenutnu grešku $e(kT)$ (razliku između željene i izmjerene vrijednosti regulisane veličine, npr. brzine vrtnje motora dobijene iz nekog senzora), a izlaz u_out predstavlja izračunatu upravljačku vrijednost $u(kT)$ koja bi se u potpunom sistemu dalje proslijedivala, na primjer, PWM modulu radi upravljanja izvršnim elementom.

Zadatak – implementacija i simulaciona verifikacija diskretnog PI regulatora

Zadatak je implementirati modul PID_regulacija prema algoritmu iz izraza (7), te njegovu ispravnost provjeriti simulacijom prije bilo kakve eventualne fizičke realizacije. Implementacija je data u Kodu 1.

Kod 1 SystemVerilog implementacija diskretnog PI regulatora (PID_regulacija.sv).

```

1 module PID_regulacija (
2     output unsigned [14:0] u_out,
3     input  signed  [15:0] e_in,
4     input  clk,
5     input  areset
6 );
7
8     parameter k1 = 10'b1010;
9     parameter k2 = 10'b001;
10
11     logic signed [15:0] u_prev;
12     logic signed [15:0] e_prev;
13     logic signed [15:0] e_prev1;
14     logic signed [35:0] d_reg;
15     logic signed [35:0] d_wire;
16
17     always_ff @(posedge clk or negedge areset) begin
18         if (areset == 0) begin
19             u_prev <= 0;
20             e_prev <= 0;
  
```

```

21         e_prev1 <= 0;
22     end else if ($signed(e_in) < $signed(16'b0)) begin
23         u_prev <= 0;
24     end else begin
25         e_prev1 <= e_prev;
26         e_prev <= e_in;
27         u_prev <= u_out;
28     end
29 end

30
31 assign d_wire = u_prev + k1 * e_in - k2 * e_prev1;
32
33 always_ff @(posedge clk) begin
34     d_reg <= d_wire;
35     if ($signed(e_in) < $signed(16'b0)) begin
36         d_reg <= 36'b0;
37     end else if ($signed(d_wire) > $signed(36'h7FFF)) begin
38         d_reg <= 36'h7FFF;
39     end else if ($signed(d_wire) < $signed(36'b0)) begin
40         d_reg <= 36'b0;
41     end
42 end

43
44 assign u_out = d_reg[14:0];
45
46 endmodule : PID_regulacija

```

Portovi i parametri

Modul ima jedan izlaz, u_{out} (15-bitna neoznačena vrijednost $u(kT)$), ograničena na opseg $[0, 32767]$, i tri ulaza: e_{in} (16-bitna označena vrijednost greške $e(kT)$), clk (takt) i $areset$ (asinhroni reset, aktivan kada je jednak nuli). Greška je predstavljena označenim brojem jer regulisana veličina može biti i veća i manja od željene vrijednosti. Parametri k_1 i k_2 odgovaraju koeficijentima K_1 i K_2 iz izraza (6), sa numeričkim vrijednostima $K_1 = 10$ i $K_2 = 1$.

Pamćenje historije (registri u_prev , e_prev , e_prev1)

Pošto izraz (7) zahtijeva prethodnu upravljačku vrijednost $u((k-1)T)$ i prethodnu grešku $e((k-1)T)$, modul mora pamtit i te vrijednosti između dva takta. To se ostvaruje registrima u_prev , e_prev i e_prev1 , koji se ažuriraju u prvom `always_ff` bloku. Pri aktivnom asinhronom resetu svi registri se postavljaju na nulu. Ukoliko je trenutna greška negativna, u_prev se prisilno postavlja na nulu – ovo predstavlja dodatnu, u osnovnom izrazu (7) nepostojeću zaštitnu granu, čija je svrha vjerovatno sprječavanje neograničenog rasta integralnog djelovanja (tzv. *integral windup*) u slučaju kada je regulisana veličina već iznad željene vrijednosti. U svim ostalim slučajevima, registri kliznu za jedno mjesto: e_prev1 preuzima staru vrijednost e_prev , e_prev preuzima trenutnu grešku e_{in} , a u_prev preuzima prethodno izračunati izlaz u_{out} .

Proračun i zasićenje (d_wire, d_reg)

Signal d_wire predstavlja neograničeni (nesaturisani) rezultat proračuna po formuli (7): $u_{prev} + k1 \cdot e_{in} - k2 \cdot e_{prev1}$. Širina ovog signala (36 bita) namjerno je znatno veća od širine izlaza (15 bita), kako bi se tokom samog proračuna izbjeglo prekoračenje (*overflow*) prije nego što se izvrši eksplicitno zasićenje. Drugi always_ff blok tu vrijednost registruje u d_reg, uz tri mogućnosti: ako je trenutna greška negativna, izlaz se prisilno postavlja na nulu; ako proračunata vrijednost prelazi maksimalnu dozvoljenu ($36' h7FFF = 32767$, najveći broj predstavljiv u 15-bitnom neoznačenom izlazu), odsijeca se na tu maksimalnu vrijednost; ako bi ispala negativna, odsijeca se na nulu. Konačno, u_out se dobija uzimanjem donjih 15 bita iz d_reg, što je ispravno upravo zahvaljujući prethodnom zasićenju koje garantuje da je vrijednost uvijek u opsegu [0, 32767].

Testbench i simulacija

Ispravnost implementacije provjerava se simulacijom, korištenjem testbench-a datog u Kodu 2. Za razliku od simulatora koji direktno izvršavaju SystemVerilog, ovdje se koristi Verilator – alat koji SystemVerilog opis modula prevodi u ekvivalentan C++ model, koji se zatim povezuje sa ručno napisanim C++ testbench-om. Ovakav pristup omogućava da se test-scenario (redoslijed i trajanje vrijednosti na ulazu e_in) opiše u vidu običnog C++ programa, dok se signali tokom simulacije snimaju u FST zapis talasnih oblika koji se naknadno pregleda u programu GTKWave – pristup koji je znatno brži za otkrivanje grešaka nego programiranje i testiranje direktno na razvojnoj pločici, i koji se preporučuje kao prvi korak provjere svake digitalne implementacije prije njene fizičke realizacije.

Kod 2 Verilator C++ testbench za simulaciju modula PID_regulacija.

```
#include <stdlib.h>
#include <iostream>
#include <verilated.h>
#include <verilated_fst_c.h>
#include "VPID_regulacija.h"

#define MAX_SIM_TIME 2200
#define CLK_PERIOD 100 // period takta u ns (posedge clk na svakih
    100 ns)
vuint64_t sim_time = 0;

double sc_time_stamp() {
    return sim_time;
}

int main(int argc, char** argv, char** env) {
    VPID_regulacija *dut = new VPID_regulacija;

    Verilated::traceEverOn(true);
    VerilatedFstC *m_trace = new VerilatedFstC;

    dut->trace(m_trace, 5);
    m_trace->open("waveform.fst");
```

```

while (sim_time < MAX_SIM_TIME) {

    if (sim_time % CLK_PERIOD == 0) dut->clk = 1;
    if (sim_time % CLK_PERIOD == CLK_PERIOD / 2) dut->clk = 0;

    if (sim_time == 0) {
        dut->areset = 0;
        dut->e_in = 0;
    }
    if (sim_time == 20) {
        dut->areset = 1;
    }

    if (sim_time == 100) {
        dut->e_in = 100; // mala pozitivna greska -> u_out treba
                        rasti
    }
    if (sim_time == 600) {
        dut->e_in = 5000; // velika pozitivna greska -> provjera
                        zasicenja na 32767
    }
    if (sim_time == 1100) {
        dut->e_in = (uint16_t)(int16_t)(-50); // negativna greska ->
                        u_out treba pasti na 0
    }
    if (sim_time == 1400) {
        dut->e_in = 10; // mala pozitivna greska -> provjera
                        oporavka
    }
    if (sim_time == 1700) {
        dut->e_in = (uint16_t)(int16_t)(-1000); // veca negativna
                        greska
    }
    if (sim_time == 2000) {
        dut->e_in = 0; // vraćanje na neutralnu gresku
    }

    dut->eval();
    m_trace->dump(sim_time);
    sim_time++;
}

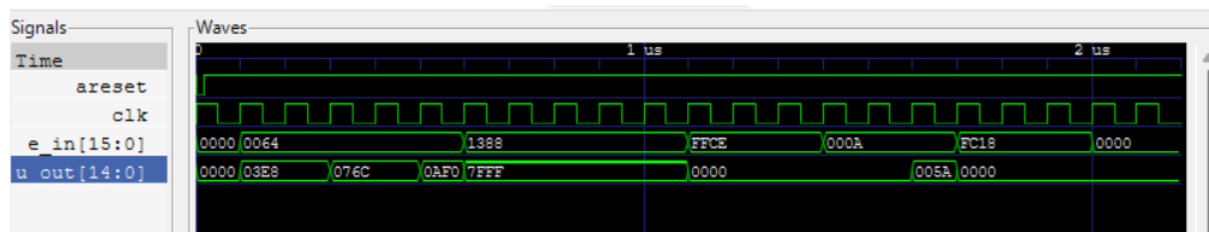
m_trace->close();
delete dut;
exit(EXIT_SUCCESS);
}

```

Testbench redom dovodi na ulaz `e_in` sekvencu vrijednosti $0 \rightarrow 100 \rightarrow 5000 \rightarrow -50 \rightarrow 10 \rightarrow -1000 \rightarrow 0$, svaku dovoljno dugo da se na izlazu ustali odgovarajuća reakcija.

Analiza rezultata simulacije

Rezultat simulacije, dobijen pokretanjem testbench-a iz Koda 2 i prikazan u programu GTKWave, dat je na slici 2. Vrijednosti signala `e_in` i `u_out` prikazane su u heksadecimalnom zapisu.



Slika 2: GTKWave prikaz rezultata simulacije modula `PID_regulacija`: signali `areset`, `clk`, `e_in[15:0]` i `u_out[14:0]`.

Heksadecimalne vrijednosti sa slike 2 odgovaraju sljedećim decimalnim brojevima (uz napomenu da je `e_in` označen broj, pa se npr. `FFCE` čita kao -50 , a `FC18` kao -1000 , u zapisu dvojnog komplementa): `e_in`: $0 \rightarrow 100 \rightarrow 5000 \rightarrow -50 \rightarrow 10 \rightarrow -1000 \rightarrow 0$; `u_out`: $0 \rightarrow 1000 \rightarrow 1900 \rightarrow 2800 \rightarrow 32767 \rightarrow 0 \rightarrow 90 \rightarrow 0$.

Ponašanje modula u svakoj fazi testa sažeto je u tabeli 1.

Tabela 1: Odziv modula `PID_regulacija` po fazama testa.

<code>e_in</code>	<code>u_out</code>	Komentar
0	0	Stanje neposredno nakon reseta.
100	$0 \rightarrow 1000 \rightarrow 1900 \rightarrow 2800$	Konstantna pozitivna greška – izlaz linearno raste za $(K_1 - K_2) \cdot e = (10 - 1) \cdot 100 = 900$ po taktu, kao što se i očekuje od integralnog djelovanja na konstantnu grešku.
5000	$\rightarrow 32767$	Velika pozitivna greška – proračunata vrijednost premašuje maksimum, pa se zasićenjem odsijeca na 32767 i tu ostaje.
-50	$\rightarrow 0$	Greška postaje negativna – izlaz se odmah prisilno vraća na nulu, nezavisno od prethodne (maksimalne) vrijednosti.
10	0 (jedan takt kašnjenja), zatim $\rightarrow 90$	Greška je ponovo pozitivna, ali se oporavak dešava tek nakon što registri historije (<code>e_prev</code> , <code>e_prev1</code>) "proguraju" novu, malu vrijednost greške kroz sebe.
-1000	$\rightarrow 0$	Ponovna negativna greška – izlaz se, kao i ranije, momentalno vraća na nulu.

Vidljivo je da se svih sedam prelaza sa slike 2 tačno poklapa sa vrijednostima navedenim u tabeli, čime je potvrđeno da modul u simulaciji reaguje onako kako je opisano.

Iz tabele 1 se vidi da modul, u pogledu vidljivog ponašanja na izlazu, ispunjava osnovni zahtjev: dok je greška pozitivna, upravljačka vrijednost raste (integralno djelovanje), veliku grešku ograničava zasićenje, a čim greška postane negativna, izlaz se bezbjedno vraća na nulu.

Testiranje

Verifikacija u ovoj vježbi izvedena je isključivo simulacijom, korištenjem testbench-a iz Koda 2, bez fizičke realizacije na FPGA razvojnom sistemu – povezivanje modula sa stvarnim izvršnim elementom (npr. DC motorom preko PWM signala) izlazi izvan okvira ove vježbe. Simulacijom je potvrđeno da modul ispravno akumulira upravljačku vrijednost pri konstantnoj pozitivnoj grešci, da zasićenje ograničava izlaz na propisani maksimum, te da se pri negativnoj grešci izlaz bezbjedno vraća na nulu.

Literatura

- [1] Haugen, F., “Tidsdiskret pid-regulator (kapittel 6, sesm3401 – styring av mekatroniske systemer)”, TechTeach / Høgskolen i Buskerud, dostupno na: https://techteach.no/fag/sesm3401/h08/pid_diskret/tidsdiskret_pid_reg.pdf